An Experience Report on a Specification-Driven, AI-Assisted Software Construction Project
Team Details (Team [18])

[Helena Honkanen] [2403510] [Solo] [100%]

Executive Summary

The project was to build a small web application with both vibe coding and specification-driven development (sdd) to compare the outcome and development of the application. The application of choice was a small expense tracker meant for personal use. The app needs to let a user add, edit, and delete expenses with the data for an expense being amount, date, category, necessity and an optional note. The app then needs to filter these expenses, let the user view a monthly summary and then chart the expenses by category. The application also needs to have a feature for the user to set a monthly budget for a category. The applications used the same stack for the application, using Python 3.15, FastAPI, SQLite and a HTML and JavaScript frontend. Both builds were also made with the same AI, being the Github Copilot inside of VS Code switched to Agent mode with the automatic model selection on, this selecting GPT 6 Luna for both the vibe and sdd version. The vibe-coded version was made by talking in plain words with conversational prompts with no written specification for the model to follow. The vibe-coded version reached a version which fulfilled all the features in just 3 prompts after an estimated 2 hours of work. The SDD build used Githubs Spec Kit and followed its workflow, being constitution, specify, clarify, plan, tasks, analyze and implement. The analyze step found six issues with the plan before the development started in the next phase, which let it fix these before it started. The application was completed in 13 prompts which took around 8 hours of work. Both builds were verified after development with the same verification plan. The verification was automated tests ran with their own pytest suites with coverage measurements, bandit security linting and then pip-audit checks for the dependencies used. After this there was a manual verification where a human tested edge-cases using the web-application and the API. After the edge cases there was also an acceptance check for the features. Using the human-in-the-loop review a GitHub issue was raised for each problem encountered during the verification.

1. Introduction & Paradigm Shift (3 Points)

1.1 Project Overview & Objectives The project idea is to create a finance tracker where one can input their spendings and make it easier to possibly cut costs or to just keep track of what they are spending and where. A user can record expenses with an amount, date, category, necessity and optional note, letting them then edit and delete these expenses as need be. The application then needs to let the user filter the expenses according to date, category and necessity and it needs to show a monthly summary with a chart of the spending for each month. The application also needs to include a feature for setting budgets per category with a limit per month. The expenses need to be saved between sessions as well. The objective of

the project was to build the same application twice so that vibe coding and SDD based coding could be compared. They were both built with the same features required, using the same technology stack and with the same AI tool.

1.2 The Vibe Coding

Vibe coding is the act of talking with an AI model with human language to produce some application or other form of software. The developer that is vibe coding is describing the thing they want to build in natural language to the model, and then the model produces this as a response to this. If there are possible changes or fixes the developer's needs or wants then they respond with this in the next prompt. I used this approach in the vibe-coded build by writing a short first prompt which included all the features I wanted but in my own words instead of with criteria. I then sent it to the Github Copilot chat which directed it to GPT Luna 6. After the model model had built the application and responded with it I checked it out by running the app, compared it to what I expected and the wrote the next prompt according to what was missing. The build using this method was completed in just 3 prompts.

1.3 The SDD Methodology

Specification-driven development is the development method where the developer makes a specification and has it as the focus of the project. The specification has the requirements, constraints and design decisions figured out before any code is written so that these can be checked before development. For the SDD build I used the Github Spec kit which is an open-sourced SDD toolkit with instructions for an AI model. I chose it since it is the toolkit that was included in the course materials and since it also integrated well with the Github Copilot model which I used. The scope of both builds was the same 7 feature list with the appropriate criteria for when it can be considered done. Both builds were kept to purely these features and any out-of-scope features that the AI may have tried implementing were excluded. The timeline of the project was: Setup phase, October 16-17, Created the repository, feature list, decided on tech stack Vibe build, October 17-21, First prompt, Prompt 1, 2 and 3 SDD build, October 22-24, Constitution, specify, clarify, plan, tasks, analyze, implement Verification, October 25-31, Automated and manual verification steps

2. Requirements & System Specification: (6 points) This is the most critical section of the report. It details the "source of truth" you constructed to feed into the AI.

2.1 Stakeholders:

End users Private individuals who want to track their spending and stay within their own budgets. They are included since they are the primary user of the application and all features are derived from what are their expected needs. Developer The developer who wants to have clear code which meets the requirements, is maintainable and testable. The developer is included since it is them who writes the specification, prompts the AI and verifies the outcomes.

2.2 Functional Requirements:

The functional requirements exist to describe what the application needs to do, so what features need to be available to the user. The functional requirements were written in the Spec Kit's specification.
The functional requirements were:

The user can create an expense with amount, date, category, necessity flag and optional note.

Amount, date, category and necessity are required fields.

Invalid amounts (zero, negative, more than two decimals) and invalid dates are rejected.

The user can edit and delete existing expenses.

Expenses can be filtered by date range, category and necessity.

The system shows a monthly summary with totals per category.

A chart visualizes spending per category, matching the summary.

The user can set a monthly budget per category.

Budget progress is shown and recalculated when expenses change.

All data persists between sessions.

Invalid input produces clear feedback to the user.

2.3 Non-Functional Requirements:

The non-functional requirements describe how well the application must perform the functionality. So, they cover the quality of the application rather than the features, such as security, reliability, usability and performance.

The non-functional requirements for this application were:
Security: all input validated on the server, not only in the browser, data-changing requests accepted only from the app's own origin, user text displayed as text, dependencies checked with pip-audit.

Reliability and data integrity: money stored without floating-point errors and data survives a server restart.

Usability: a first-time user can add an expense without instructions. There needs to be validation errors shown next to the form.

Maintainability and testability: every feature covered by automated pytest tests.

Performance: no formal target was set. A target of one second for 10,000 records appeared in the generated plan but was removed during analysis, because it was not part of the original requirements.

2.4 Technology Stack
Language: Python 3.15

Language I am most comfortable reading and reviewing, which made reading AI-generated code easier.

Backend: FastAPI

Only API framework I have used previously. It also has automatic API documentation which was useful for testing the API directly.

Database: SQLite

Used because of the small scale of the app. A single local file is enough for a single-user app.

Front end: HTML, CSS and JavaScript

No framework, so the codebase stays small enough to review. The frontend did not need to be fancy, just fulfill the requirements.

Testing and auditing: pytest, pytest-cov, bandit, pip-audit

Standard Python tools for tests, coverage, security linting and dependency checks.

2.5 Interface & API Specifications
For the SDD the API was specified before it was implemented in the /specs/contracts folder

Conventions defined in the contract

Money is exchanged as decimal strings in euros, never as JSON floating-point numbers, so amounts cannot pick up rounding errors.

Dates use YYYY-MM-DD; months use YYYY-MM.

Validation errors return 422 with a field-specific message. This covered malformed JSON, missing fields, unknown categories, invalid dates or months, and amounts that are non-positive or have more than two decimals.

Missing records return 404. A successful delete returns 204 with no body.

Data-changing requests are accepted only from the app's own origin.

Responses never expose internal database paths or stack traces.

2.6 How these requirements and specifications were created (using SDD AI tools)

The specification was produced with Spec Kit's commands in GitHub Copilot, with me reviewing and approving each document before moving on:

Constitution: I gave the stack and quality principles for the application.

Specify: I gave the feature list with its “done when” criteria. The spec Kit generated spec.md with user stories, numbered requirements, success criteria and a quality checklist.

Clarify: the agent asked questions about unclear points. For example, it asked whether data export was needed, and I marked it out of scope since it was out-of-scope.

Plan: the agent produced the technical design (plan.md, data-model.md, API contracts and quickstart.md).

Tasks: the plan was broken into 27 numbered tasks in six phases.

Analyze: a consistency check across all documents found six issues, including two of high severity. I decided how each should be resolved, the documents were updated, and a second analysis came back clean.

3. Vibe coding and your project

Since I did this project as a solo developer there was no necessary division of tasks. I acted as the sole prompter, tester and reviewer, which in hindsight definitely was not ideal. As for the vibe coding I did not split up the task in parts or plan it too much ahead and just created the application from one master prompt which had the features on it in plain language. The result of that prompt was instead then improved upon by writing follow up prompts to the AI. Both builds had the same rule of stopping as soon as all features were met, so the implementations could also vary a bit if they just met the base criteria.

3.1 Prompting Framework

The framework most resembled a zero-shot framework with role prompting since I only gave the prompts in plain language without any examples or chain-of-thought instructions but with a role for the expected quality and point of view for the task. The fix prompts did not include the role prompting part since they continued the same conversation. The agentic part for the vibe-coded version came purely from Copilots agent mode which turned these prompts into a planned application. The reason for this framework was that I truly wanted to compare the arithmetic thoughtful method of SDD compared to the purest simplest form of development with AI.

3.2 Master Prompts / System Prompts:

My first prompt was a master prompt which included all of the features and rules for the application. I took my earlier feature list and rewrote it so that it did not include all of the ready criteria for the AI.

Prompt 1 Prompt: You are an expert software engineer tasked with making an application for managing and tracking expenses with python, FastAPI and SQLite. The frontend should be a web page. The application must allow for adding, editing and removing expenses, and the expenses should have an amount, date, category, information if it was a necessity and an optional note about the expense. It must also include a feature for filtering the expenses and it

must have a monthly summary for the expenses by category. The application must have a feature for setting a monthly budget for the categories and it also needs a separate spending chart where the categories can be shown.

Expected answer/result: A working base version which would fill the basic requests and be built with the correct tech stack.

Outcome of prompt: A base version which does look professional and seems to deliver on most basics, but it is missing some features and has some flaws. Main features that were directly asked for that are missing or broken are: Filtering by the date, for this to work there of course also needs to be a monthly or global list somewhere for the expenses, the budgets can't be edited in any intuitive way and instead need the user to resubmit the budget for the category, the budgets are also missing a delete button and feature completely since they can't be re-submitted with the value at 0, this is also one thing the AI has invented since a user may want a total 0 budget on something a certain month. A feature that was not explicitly asked for but that the AI decided, was to show all of the categories in the budget section, even though they did not have budgets set. So, once a category got a budget all of the other categories were also listed on the budgets list even if they had no budget set.

Prompt 2

Prompt: There are some parts missing, I wanted filtering by date, I need the budgets to have a button to edit and remove them, and the budget list shows categories which don't have budgets, the user also can't add a budget of 0 for a category.

Expected answer/result: Fixes to all requested issues with the possibility of an additional feature by the AI

Outcome of prompt: A very clean fix to all of the issues listed, date filtering was added with an additional clear dates button that was not asked for but nice to have. The budgets were also fixed to only show categories with budgets and now budgets with a value of 0 could be added. One thing however which was noticed when testing this version was that the selection of category when adding an expense did not suggest existing categories, but the user instead had to remember them and make sure the letters were capitalized the same. One obvious missing part was also the visual spending chart which should show the different categories and their respective parts of the total.

I wrote prompt 2 to fix all the errors I noticed after prompt 1, but this still did not meet every criteria, so there was a third prompt. Before writing the third prompt I raised GitHub issues for the things that were left to fix.

3.3 Prompt Iteration Log

Prompt 1.

The biggest failure for the first prompt was the missing date filter along with there being no full list of expenses to filter. This however can be blamed on my initial prompt and the fact that I purposely left it vague at certain parts to see how the AI implements it. I never mentioned date filtering, so it was not included, but in the fix prompt 2 I revised on this and requested this filtering which was successfully added. The AI also included a unwanted feature for the budgets category, since it listed all of the existing categories in the budget section instead of only listing the ones with existing budgets. The budgets were also set in stone after they were implemented, and could not be removed at all, so I requested a fix for this in the second prompt. The budgets could also not have 0 as a value, which was hallucinated by the AI since someone may want to set a 0€ budget on something for that month.

Prompt 2.
The second prompt fixed the original issues or missing features that the first one had, but it had implemented its own clear dates button for emptying the date filtering. It also still had missed out on the visual spending chart for the spending of the different categories and their parts of the total.

4. Specification Driven Development and Your Project (7 pts)

4.1 SDD Tool-kit

The SDD tool-kit used was the GitHub Spec Kit, which is the open-source toolkit that is provided by GitHub which adds the SDD commands to the AI agent. I chose for a few reasons: it was the spec-kit that was mentioned in the course materials, it integrated with GitHub Copilot which was the AI agent I was going to use and it enforced a workflow that had artifacts and documents that would come in handy once the project was done and this documentation was up next.

4.2 Development Environment

I used macOS with VS Code as my IDE since that is what I am most familiar with. I used the GitHub Copilot Chat with agent mode (which routed to GPT 6 Luna) purely because it integrated with the spec kit and it felt as the easiest way to compare the build to the vibe coded version. I used python 3.15 for both builds to make sure that they had the same version to build with so that there could be as little differences as possible.

4.3 Deriving the requirements using the IDE/coding agents

The requirements were derived step by step through Spec Kit commands, with each step producing a document that I reviewed before moving on:

Software Requirements Specification: /speckit-specify turned my feature list into spec.md. It included user stories with priorities, acceptance scenarios, numbered functional requirements (FR-001–FR-011), success criteria (SC-001–SC-005) and a requirements-quality checklist. /speckit-clarify then asked targeted questions about unclear points, and my answers were written back into the specification.

System constraints: the stack was declared in the constitution, server-side validation of all input, and automated pytest tests for every feature. Every later step included a "constitution check" against these rules.

Interfaces: /speckit-plan produced the API contracts, the data model and the overall design.

Acceptance criteria: the "done when" criteria from my feature list became acceptance scenarios and success criteria in spec.md, and quickstart.md turned them into a manual walkthrough, which later revealed the unfinished UI in phase 6.

Compliance requirements: compliance with the constitution was done with /speckit-analyze, which checked that every requirement had tasks and that nothing contradicted the constitution.

4.4 SDD Workflow and Implementation

I followed the spec kit's standard workflow with these decisions of my own: I did it one phase at a time, and I committed after each step. The workflow was: Constitution, Specify, Clarify, Plan, Tasks, Analyze (Until consistent), Implementation one phase at a time until done. The spec kits default plan implements all phases at once, but I wanted to make sure that I had the possibility to see the phases of it (even though it took a lot longer).

4.5 Prompts Describe and include the prompts that you have used, add your reasoning for the prompt, what you expected the prompt would do, what did not go according to plan, and how you modified the prompt later. Document the prompts you used during AI-assisted development (SDD) and explain how they evolved throughout the project. For each significant prompt, include the original prompt text, the purpose of the prompt, and your reasoning for using it. Describe what you expected the AI to generate, evaluate whether the output met your expectations, and discuss any issues encountered (e.g., incorrect, incomplete, or inefficient code). If the initial prompt did not produce the desired result, explain how you refined or modified the prompt, why those changes were made, and how the revised prompt improved the outcome. Include representative prompt-response examples where appropriate, and reflect on the importance of prompt engineering in achieving accurate and reliable AI-generated software artifacts.

In the SDD build, most prompts were Spec Kit commands. The specification documents carried the actual requirements, so a prompt's job was to trigger a step and add only the decisions the documents didn't contain yet. Each prompt below lists its text, purpose, expectation and outcome.

P0.1 / P1: Constitution

Use Python 3.15 with FastAPI and SQLite with the frontend being HTML and JavaScript. Keep it simple, no frameworks. All user input needs to be validated and every feature must have automated tests with pytest.

Purpose and reasoning: fix the stack and the non-negotiable quality rules before any requirement existed, so every later step would be checked against them.

Expected: no code, only an updated constitution.md as context for later steps.

Outcome: the first run (P0.1) failed without making changes. Spec Kit had been initialised with PowerShell scripts, which don't run on macOS. After re-initialising with shell scripts, the identical prompt (P1) produced the constitution. The prompt itself didn't need changing; the failure was in the environment.

P2: Specify

/speckit-specify An expense tracker that helps the user see what their money goes to and helps them cut costs. Feature list: The application needs to have the possibility to add expenses with the data being: Amount spent, date, category of purchase, option to select it as a necessity, option for a note. ... The application also needs to have a separate spending chart where the different categories can be visualized.

Purpose and reasoning: give the full feature list, including validation and persistence rules, but no technology. Unlike the vibe master prompt, this was meant to be complete, because the specification is the source of truth.

Expected: a new branch with spec.md, and a list of points needing clarification.

Outcome: spec.md captured every requirement, which I checked by reading it against the feature list. No points were marked as needing clarification. Contrary to expectation, no branch was created: in the current Spec Kit version branch creation is an optional extension. I created the branch by hand, named after the spec folder.

P3.1–P3.4: Clarify

/speckit-clarify A A suggested

Purpose: resolve decisions the specification left open, before planning.

Expected: questions one at a time, with options.

Outcome: three questions, each with options and a recommendation:

User scope: single user without login (A), single user with login, or multiple users. I chose A.

Data export for backup: none (A), CSV export, or automatic backups. I chose A, keeping scope equal to the vibe build.

Currency: I accepted the recommended euro.

Each answer was written into the specification, and the requirements checklist still passed. None of these three decisions came up in the vibe build, where the AI made them silently. Here they became explicit, documented choices.

P4: Plan

/speckit-plan Use the defined stack in the constitution. Keep the implementation simple with one FastAPI app, a SQLite database and a frontend made with HTML and JS.

Purpose and reasoning: produce the technical design. The prompt referred to the constitution instead of repeating it.

Expected: plan.md listing the stack, matching the constitution.

Outcome: the agent used a subagent to research simple designs, then wrote plan.md, research.md, data-model.md, the API contracts and quickstart.md. The stack matched. The plan also introduced a performance target nobody had asked for, which went unnoticed until the analyze step.

P5: Tasks

/speckit-tasks

Expected: tasks.md with tasks and tests, to review whether the AI understood the work.

Outcome: 25 tasks organised by user story, each story with its own tests. On review, the tasks covered every story.

P6: Analyze

/speckit-analyze

Purpose: find inconsistencies between the documents before any code existed.

Outcome: six findings, including two of high severity:

I1: contradictory behaviour for a reversed date range.

A1: an untestable criterion requiring 90% of first-time users to succeed.

C1: an endpoint with no task.

C2: the invented performance target.

U1: unnamed category list.

U2: unclear audit command.

Several were hallucinations that had passed through the plan and tasks, unnoticed by me, which shows the value of an automated cross-check.

P7: Fix the analysis findings

Apply the following fixes to the files: I1, a reversed date range needs to return 422 with an error message. A1, replace SC-001 with the requirement “a first-time user can add an expense to the app without further instructions”. C1, add the implementation and test tasks for GET/api/categories. U1, List the category slugs in the spec and T006. C2, This was not apart of scope so remove the performance target from the plan. U2, use pip_audit -r requirements.txt

Purpose and reasoning: the agent had offered to propose fixes. I made each decision myself and listed exactly what to change, so the AI could not make scope decisions on my behalf.

Expected: changes to only the listed items, with a confirmation.

Outcome: exactly the requested fixes, nothing more.

P8: Re-analyze

/speckit-analyze

Outcome: a clean report, with no findings and full coverage. The specification was ready for implementation.

P9–P14: Implement, one phase at a time

/speckit-implement Implement the first phase from the tasks.md and stop after phase 1.

5. Verification and Validation (6 pts) Since AI code can contain subtle bugs and security vulnerabilities, document how you verified the output.

5.1 V&V Strategy

Verification happened at two different times: during each build, by the AI agent and by me, and after both builds, by applying one shared verification plan identically to each build's final version.

During the builds:

Vibe build: human-in-the-loop only. After each prompt I ran the app and compared it with my feature list. The agent also tested in a browser on its own, but I never asked for tests, so any tests it wrote were its own initiative.

SDD build: test-first development driven by the specification. The constitution required automated tests for every feature, so tasks.md contained test tasks for each user story, written before the implementation. Failing tests that later pass appear in phases 3 and 4. After each phase I reviewed the ticked tasks, ran the tests and checked the app. In the final phase the agent ran the quickstart.md walkthrough as an end-to-end acceptance test, plus pip-audit as required by the plan.
Threats to validity:

Order effect: the vibe build came first, so lessons from it could have influenced the SDD build. Verification was run only after both builds were finished, so audit findings couldn't leak into the SDD specification.

Workspace contamination:

The first vibe run happened while the specification documents were in the workspace; it was discarded and restarted.

The SDD agent saw the vibe build's leftover database schema in phase 5.

Different tool versions: the SDD build's dependency fix raised pytest to 9.x, while the vibe build ran on 8.4.2.

All manual checks were done by me who knew the expected behaviour.

5.2 Code Audit Log
Vibe, Spending chart missing although explicitly request, found by me while testing, Fixed by prompt P03

Vibe, Budgets could not be edited or removed, AI invented a "budget above 0" rule, found by me while testing, Fixed by prompt P02

Vibe, Free-text, case-sensitive categories created duplicates, found by me while testing, Fixed by prompt P03

SDD, Spec contradiction on reversed date ranges, untestable usability criterion, invented performance target, found by AI (/speckit-analyze), fixed in prompt 7

SDD, Expense form never worked in the browser in phases 3–5, tests covered only the API, found by AI, Fixed by the agent in phase 6

SDD, Data-changing requests accepted from other local origins and ports, found by AI, fixed by AI

SDD, Vulnerable pytest version, blocked by an unrequested version limit, found by pip-audit, fixed by AI

SDD, SQLite connections not closed, found by pytest, not fixed

Both, Six-digit years accepted by date inputs, budget month selector showed a format error, found by me in manual testing, not fixed

Both, Starlette/httpx deprecation warning, found by pytest warnings, out of scope, no functional impact

5.3 Test Execution Results

Check	Vibe	SDD
Tests run / passed / failed	3 / 3 / 0	12 / 12 / 0
Line coverage	92% (157 statements, 12 missed	84% (311 statements, 49 missed)
Warnings	1 (deprecation)	16 (1 deprecation, 15 unclosed database connections)
bandit: high / medium	0 / 1 / 0 (investigated: false positive)	-
pip-audit: vulnerable packages	1 (pytest 8.4.2)	-

Evidence: docs/vibe/ and docs/sdd/.

Both versions passed all manual tests, some in different manners, such as the long note on an expense. The vibe coded version had a limit on the note length while the sdd did not seem to have one, which made the view after adding it look weird (longnote.png on the sdd branch).

Coverage is not directly comparable. The SDD app has twice as much code and four times as many tests. High coverage from only three vibe tests shows that coverage measures which lines ran, not if edge cases were checked.

The SDD build's phase 3 tests passed while its UI didn't work, and the vibe agent's self-testing never noticed the missing chart. The defects that mattered most were found by manual testing.

6. Project Management & AI Toolchain Workflows

All work was done in the GitHub repository honkanenhelena/sdd-course-project, so the commit history and issues document the whole process.

6.1 Version Control & Prompt Integration

main, Shared starting point (feature list, tech stack, constitution), verification plan, and at the end all logs, evidence and specifications collected for the report

vibecodedbranch, The vibe build, with Spec Kit and the specification documents removed

001-expense-tracker, The SDD build, named after its spec folder

The commits start with either prompt or documentation, vibe or verification.

Issues and labels: defects that survived a fix attempt, and every verification finding, were recorded as GitHub issues with labels for the build (vibe, sdd) and the type.

6.2 Team Contribution Breakdown

This was a single-person project, so I held every role (100% contribution). No code was written by hand in either build. My work shifted entirely to specifying, prompting, reviewing and verifying, and the balance between these roles differed clearly between the two approaches.

Specifier

Vibe: the requirements existed only in my head and in the chat prompts
SDD: feature list, constitution, clarify decisions, and reviewing spec, plan and tasks before any code existed

Prompt engineer

Vibe: the master prompt and correction prompts carried all requirements

SDD: prompts were mostly Spec Kit commands, the key ones carried decisions

Orchestrator

Vibe: Approving agent commands, committing per prompt

SDD: Running the workflow step by step, limiting implementation to one phase per run, managing branches and tags

Verifier

Vibe: Testing in the browser after each prompt, then the full verification plan

SDD: Reviewing each phase’s tests and ticked tasks, then the full verification plan, including investigating tool findings

As one can see from these differences, the largest difference was that most of the effort in the SDD was before the app was made, while the vibe coding example needed more effort in the verification and correction phase after the first version was made.

7. Reflections on Specification-Driven Development (5 points)

7.1 Benefits of AI-Assisted SDD

I thought that the planning phase was the most significant improvement on manual coding. I thought this since this allowed me to be thorough in my planning and let me make a solid plan on all necessary parts, and then using just that plan it made the application. The fact that problems could be resolved before it was in actual code was also a big improvement on the time that usually gets spent chasing bugs in code. I would say that the speedup was more so on getting the project in more or less one shot than in time, but this was also due to it needing heavy documentation for this task and the fact that I split up the phases. I believe that the speedup could be very significant when making a larger and more complex system than this expense tracker.

7.2 Core Bottlenecks and Challenges

One issue the AI had was in phase 3 where the AI only tested the API for the expenses, but the form didn't actually work in the browser. It did fix it in phase 6, so in a default spec kit the phase prompt would have created and fixed the problem in the same prompt, but it still showed that it can make errors in the testing. It also added things that were not requested, such as the performance target of 10 000 users. While these issues were small in my simple expense tracker I do believe that they are proof of what could happen in a larger project. With similar issues at a larger scale it is impossible to know, based on this experiment, if they would create more issues and maybe not fix themselves.

7.3 Lessons Learned & Best Practices

The biggest surprise was that vibe coding was much faster to a working app, and its failures were in what the prompt left out. Everything I specified clearly was built, except the chart. The rest of the problems came from things my prompt only implied. The AI filled those gaps with its own choices, some good ones and some not so good.

SDD moved the time and effort from fixing stuff to deciding how it's built. The vibe build needed two correction prompts after the code existed. The SDD build needed none, because its corrections happened in the documents. But I believe that this also proves that SDD has it's value in a larger codebase rather than for such a small simple project.

The most serious defect in SDD passed through three phases of tests because the specification tests covered the wrong layer.

Neither approach replaces end-to-end testing. In both builds the defects that mattered most were found by a person, or a walkthrough, using the app, not by unit tests or by the agent's own checks.

Best practices for future students

Write acceptance criteria that include the user interface, and require an end-to-end check after every user story.

Make every decision yourself when the AI asks.

Run the analyze step until it is clean. It caught a lot of problems which would have shown up later which just would have led to the SDD being more like the vibe coded one in spending time on fixing it rather than planning it.

Implement one phase at a time and commit after each, so a defect can be traced to the step that introduced it.

Review every diff the agent makes, including files you didn't ask to touch.

Verify findings instead of trusting them blindly. Bandit's SQL injection warning was a false positive on inspection.

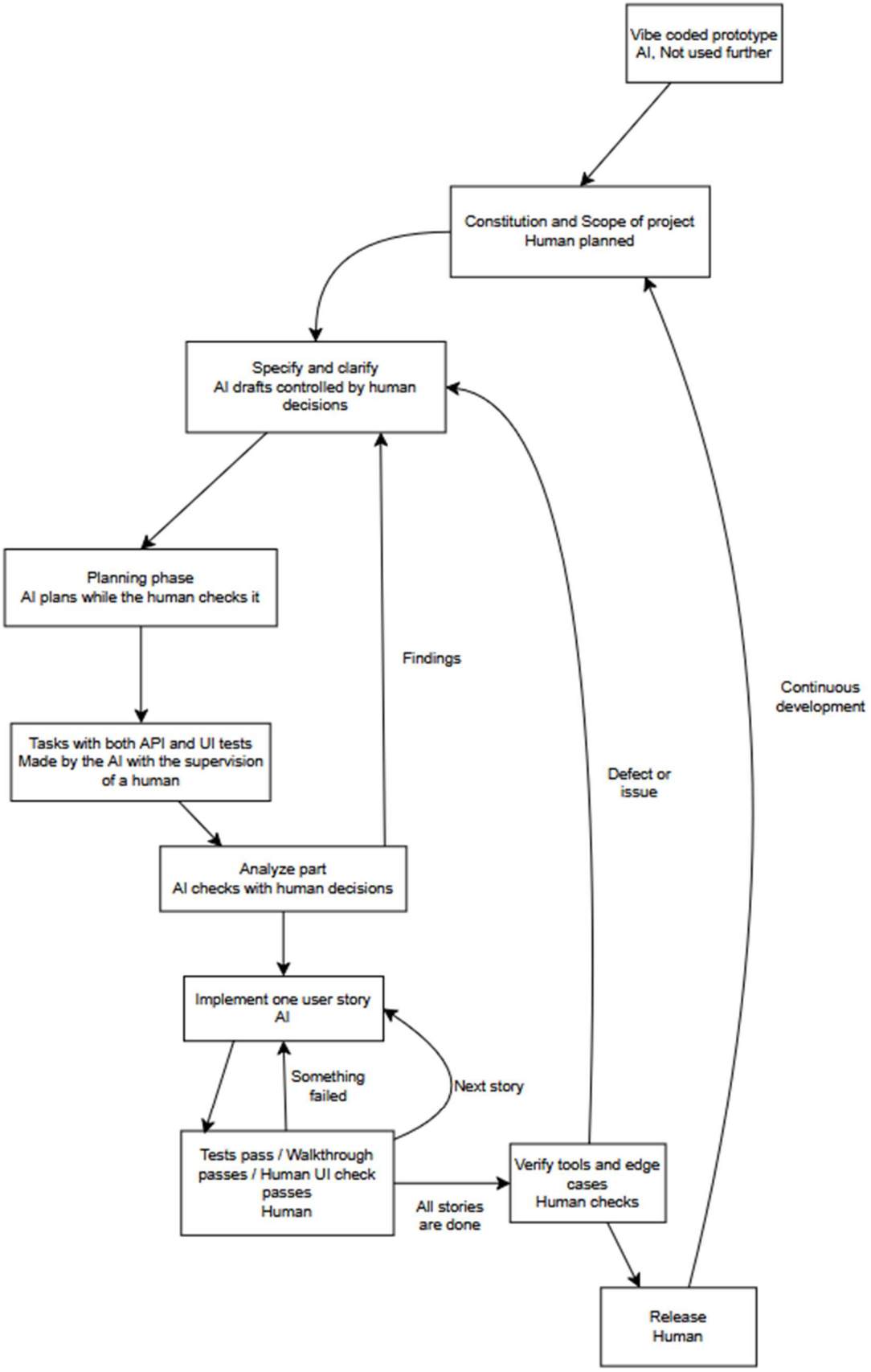
Log each prompt's expectation before running it.

Keep experiment workspaces clean. Check git status and ignored files before switching branches and keep documents the AI shouldn't see out of its workspace.

Balanced reflection

This project introduced me to a new to me way of using AI tools in development. Letting the AI write all of the code in both builds based on either purely what one prompt said or what was documented was interesting. The differences in the implementations and how the development worked was the most important part to me. Vibe coding to me seems better for getting a quick prototype for an idea or for a pure simple project such as this. SDD seems much better suited to a larger problem or to a system that needs to be correct and secure. I do believe that it still produced a good application for the requirements given, but compared to the vibe coded version I did not see the value in using it for such a simple program which was completed in a significantly shorter time with vibe coding.

7.4 . Proposed Specification-Driven Development Workflow



The first phase is the only part where vibe coding comes in. It is used for getting an initial idea going and a vision planted in the users mind on what could be and how it could shape up. The

second phase is then for fixing rules and boundaries, so choosing a tech stack and what the application should do and what it shouldn't. The third phase turns the scope of the previous phase into testable requirements. The fourth part is about the design within those constraints, which needs a human to check it. The tasks part breaks up the work into verifiable steps but it needs to have checks for both the API and the UI (unlike my version). The analyze part is the first major part which can set the progress back. It catches inconsistencies before it goes into code and if there are any then it goes back to the specify and clarify step (unlike my version where I fixed these myself and input it to the AI). If the analyze is clean then the next step is the implementation of a user story with the AI. This then leads to a test phase of the user story implementation which either fails, goes back to implement the next user story or then when all stories are complete goes to the possibly final step before release. This step is the verification of the tools and edge cases which is checked by a human. If there are defects or issues at this phase then the workflow goes back to the specify and clarify step so that possible tool defects can be changed or so that edge cases can be considered in the application. After this the application is released and continuous development takes it back to the start so that the process can be done for further development.